TECHNICAL WHITE PAPER · THE AGRI-TOKEN EXCHANGE

AgriBridge

Technical white paper: tokenomics, contracts, oracle and security

VERSION	CONTRACTS	NETWORK	STATUS
1.0, October 2026	Solidity ^0.8.24, Foundry	Ethereum Sepolia (testnet)	Prototype, not audited

Abstract

AgriBridge tokenizes graded agricultural commodities held in partner warehouses and makes them usable as collateral and as tradable stock. A multi-signature **Safe** is the single verifier that mints ERC-1155 crop tokens per lot (1 token = 1 kg). A USDC lending pool lends against those tokens at up to 50% of their value **at the loan's end date**, with continuous decay of crop value built into every valuation, partial liquidation at 80% loan-to-value, and a keeper that settles positions from a reserve. A marketplace trades the same tokens. Prices come from an oracle that combines Chainlink Functions (world prices) with a reporter quorum (local prices) behind a move cap, staleness limits and a circuit breaker. This paper specifies the token model, the contract architecture, the oracle design, how price volatility is absorbed, and the security mechanisms and audit scope. All figures are taken from the deployed code.

1. Tokenomics

AgriBridge has no speculative or governance token. Every token is backed one-to-one by something real: crop in a warehouse, or a claim on the lending pool's dollars. Settlement is in USDC (a play-money DemoUSDC on the testnet).

Token	Standard	Supply rule
Crop token	ERC-1155 (CommodityToken), id = lot id, 18 decimals = 1 kg	Minted only by the registry when the Safe approves an intake, for the measured weight. Burned only by the warehouse desk when the Safe confirms the goods left. **Supply of a lot always equals the kilograms in store** (fuzz-tested invariant).
Pool share	ERC-20 + Permit (AgriShareToken), 6 decimals	Minted and burned only by the lending pool on deposit and withdrawal, at shares = assets × totalShares ÷ totalAssets, with virtual shares against first-deposit inflation.
USDC	ERC-20, 6 decimals	External. The pool, market and desk hold it; the pool tracks its own cash, so unsolicited transfers never change the share price.

Table 1. Tokens and their supply rules.

Value flow	Rule
Borrower interest	Utilisation-based: 5% a year at 0% lent, rising linearly to 13% at the 80% kink, then steeply to 23% at 100%. Accrues every second via a borrow index (scaled debt).
Investors	Receive 80% of interest as a rising share price: share value = (cash + debt − reserves) ÷ shares. Supply rate = borrow rate × utilisation × 0.8.
Loss cushion	20% of interest (reserve factor, capped at 50%) accrues to reserves, which absorb bad debt before investors and fund keeper settlements.
Settlement reward	A liquidator repays the debt and receives collateral worth debt × 1.05; the 5% is the incentive; the rest of the crop returns to the borrower.
Market fee	1% of each sale (adjustable, capped at 5%) to the treasury, held by the Safe.
Storage fee	Per tonne per month, charged pro rata by the day to whoever withdraws, paid to the warehouse fee recipient.
Clearance	Expired crop is bought by the protocol at 30% below reference from a fund the Safe tops up, then resold to feed buyers; proceeds go to the treasury.

Table 2. Where value goes.

2. Smart contract architecture

Nine contracts (about 3,300 lines of Solidity, built on OpenZeppelin AccessControl, Pausable, ReentrancyGuard and ERC-1155/ERC-20) separate settings, custody records, tokens, prices and the three applications. Contracts are immutable; references between them are set once at deployment. Every admin and verifier role is held by the Safe, and the deployer renounces all roles at the end of deployment, which the deploy script audits.

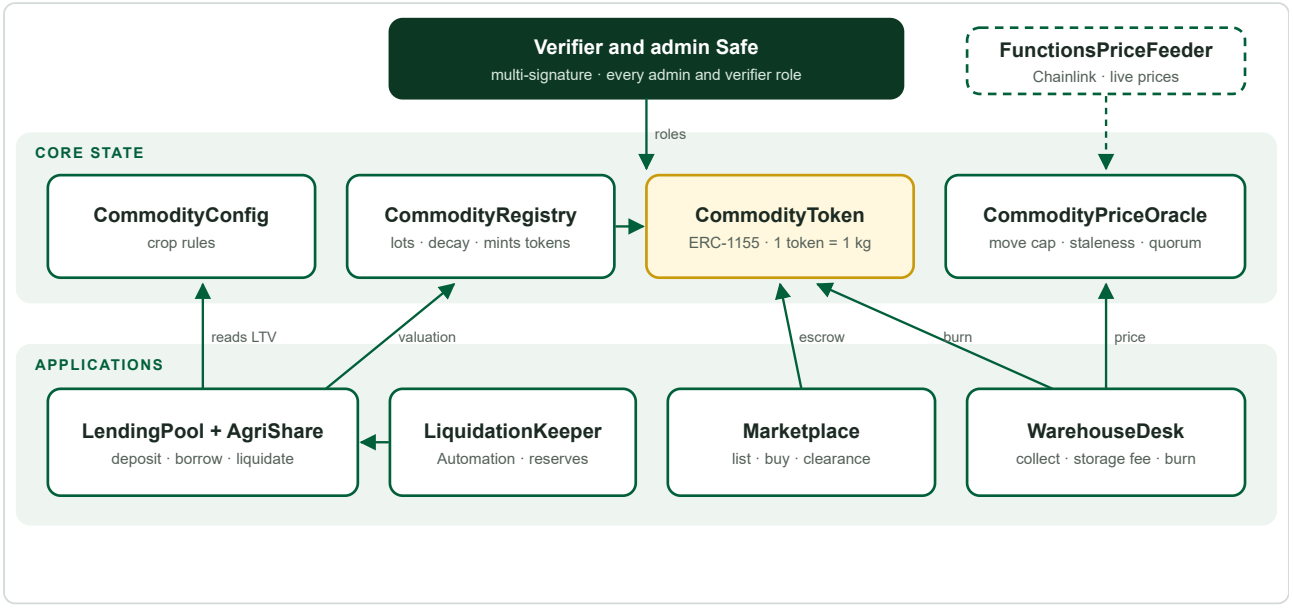


Figure 1. Contracts, the roles that drive them, and how they read each other.

Contract	Responsibility	Key functions
CommodityConfig	One row per crop: grade factors, days to grade B / C / expiry, max LTV, liquidation LTV, basis cut, storage fee, price source, active flag.	addCommodity, updateCommodity
CommodityRegistry	Warehouses (capacity enforced) and lots: intake, verification with evidence hash, freezing, the decay curve and valuation factor.	requestIntake, approveIntake, valueFactorAt, valuationFactorBps
CommodityToken	ERC-1155 crop tokens with on-chain metadata; blocks user-to-user transfers of frozen lots.	mint, burn (role-gated)
CommodityPriceOracle	Per-crop prices (8 decimals): move cap, staleness, reporter quorum, override, pause.	setPrice, submitLocalPrice, forcePrice, getPriceFresh
FunctionsPriceFeeder	Chainlink Functions client and Automation job that pushes world prices to the oracle (deployed when a router is configured).	requestPrices, performUpkeep
LendingPool	USDC deposits, shares, borrowing against lots, interest, liquidation, reserves.	deposit, withdraw, borrow, repay, liquidate
AgriShareToken	ERC-20 pool shares.	mintShares, burnShares (pool only)
LiquidationKeeper	Chainlink Automation-compatible: finds liquidatable loans and settles up to five per run from reserves.	checkUpkeep, performUpkeep
Marketplace	Fixed or follow-the-market listings, partial buys, bulk discounts, fees, clearance.	list, buy, sellExpired
WarehouseDesk	Pickup or delivery requests, storage fees, burning on confirmed release.	requestWithdrawal, confirmRelease

Table 3. Contracts.

Role	Holder	Allows
DEFAULT_ADMIN	Safe	Crop rules, warehouses, fees, reserve factor, pause, granting roles
VERIFIER_ROLE	Safe (exactly one holder, enforced)	Approve or reject intakes, which mints crop
CUSTODIAN / CLEARANCE	Safe	Confirm releases; list clearance stock
PRICE_UPDATER	Safe, and the feeder when deployed	Push prices subject to bounds and move cap
KEEPER_ROLE	LiquidationKeeper	Settle loans using reserves
MINTER / BURNER, CUSTODY / PROTOCOL	Registry / WarehouseDesk / Pool, Market, Desk	Mint crop; burn it and record the release; move frozen tokens back to owners
REGULATOR_ROLE	To be assigned	Freeze or unfreeze lots and warehouses

Table 4. Access control.

3. Oracle integration strategy

Loans and the market read only the oracle, never market trades, so borrowed capital cannot move a valuation. The oracle stores a price per crop in USD per kilogram with 8 decimals and two independent paths feed it:

- **World-price crops** (cocoa, rice, maize, soybeans): FunctionsPriceFeeder runs a Chainlink Functions job that queries several providers and returns the **median**, requiring at least two answers. Chainlink Automation triggers it, rate-limited by a minimum request interval. Fulfilment never reverts on bad data; it skips a crop instead.
- **Local-price crops** (cashew, yam): up to 7 approved reporters submit prices; a price counts when **two fresh reports agree within 5%** of their median, so an outlier is outvoted.
- **Bounds:** every price must lie between $0.01 and $1,000,000 per kg.
- **Move cap:** a change over 10% is held, not applied, until a later update at least 10 minutes on confirms it; a one-off glitch is dropped when the next update is normal.
- **Staleness:** each crop has a heartbeat; getPriceFresh reverts on a stale price, which blocks new loans and liquidations rather than acting on old data.
- **Circuit breaker and override:** pausing the oracle stops every valuation; the Safe can force a price to correct a feed, even while paused.

Festival deployment. Prices are set at deployment from current world prices with a long heartbeat, and changed by the Safe. The feeder and reporter paths are built and tested (13 feeder tests with a mocked router) but switched off until price providers and reporters are chosen.

4. Handling real-time price volatility

Volatility is absorbed in layers, from the valuation itself to the final loss backstop. The collateral value of a lot is recomputed on every read as:

> **collateral value = oracle price × kg × ageing factor(t) × (1 − basis cut)**

- **Valued at the end date.** Borrowing limits use the ageing factor on the loan's maturity, so predictable decay never pushes a loan underwater.
- **A 30-point buffer.** Loans start at up to 50% LTV and settle at 80%, so the price must fall roughly 37.5% from the opening valuation before settlement.
- **Filtered prices.** The move cap and the reporter quorum stop a single bad print from triggering liquidations; staleness pauses action rather than using old data.
- **Partial settlement.** Only collateral worth debt × 1.05 is taken; the rest returns to the farmer. If collateral is worth less than that, all of it is taken and the shortfall is bad debt.
- **Backstops.** Anyone may settle; if nobody does, the keeper settles from reserves; bad debt hits reserves before investors.

Step (500 kg grade A cocoa, $1,000 advance)	Oracle price	Collateral value	Loan-to-value
Advance taken, healthy	$5.85/kg	$2,486	40%
Settlement threshold	$2.94/kg	$1,250	80%
Oracle update: price falls	$2.50/kg	$1,063	94%, at risk
Settlement: 494.12 kg taken for $1,050; 5.88 kg returned	$2.50/kg	n/a	loan closed

Table 5. An oracle-triggered collateral adjustment, as executed in the demo (values after the 15% basis cut, ageing ≈ 100%).

With live feeds, the same 57% fall would arrive in confirmed steps under the move cap; in the festival deployment the Safe's override applies it at once, which is how the demo shows the full path from oracle update to settlement.

5. Security mechanisms

Threat	Mechanism (each backed by a named test)
Reentrancy	nonReentrant on every function that moves USDC or tokens; checks-effects-interactions; repayment takes USDC before returning collateral (attack test: re-entering during collateral return reverts the repayment).
Flash loans	No on-chain market prices; internal cash accounting ignores donations; virtual shares defeat first-deposit inflation; same-block deposit and withdraw refused; continuous accrual removes repayment sandwiches.
Oracle manipulation	Median of providers, 2-of-3 quorum within 5%, 10% move cap with 10-minute confirmation, bounds, staleness, circuit breaker.
Fake or duplicated collateral	Single verifier role (a second holder is rejected); evidence hash per lot; warehouse capacity enforced; burn on release keeps supply equal to stock.
Privileged keys	All admin roles on the Safe; deployer keeps no roles (deployment test); demo script refuses known-leaked keys.
Liveness of settlement	Permissionless liquidation plus the keeper; a keeper run with too little gas reverts instead of silently skipping loans.

Table 6. Threats and mechanisms.

Testing. 262 Foundry tests: unit and integration suites per contract; fuzz properties (a borrow never exceeds the limit; settlement never takes more than debt + 5%; a deposit never moves the share price; token supply equals stored stock; the decay curve is monotonic); an attack suite (reentrancy, donation, first-deposit inflation, same-block withdrawal, repayment sandwich, oracle jump); a rehearsal of the full lifecycle on a fork of Sepolia acting as the real Safe; and 30 browser tests that drive every role through the app against a local chain, run in CI on every push.

6. Audit considerations

- **Scope:** the nine contracts in src/ (about 3,300 lines) plus the deployment and role hand-over in script/.

- **Valuation arithmetic:** decay interpolation at grade boundaries, basis and LTV rounding, maturity-based limits, unit scaling between 8-decimal prices, 18-decimal kilograms and 6-decimal USDC.
- **Liquidation edge cases:** collateral between debt and debt × 1.05, overdue loans, frozen lots, expired collateral, partial lots, keeper batching and gas.
- **Centralisation:** forcePrice, pause, crop-rule changes and reserve withdrawals are Safe-only and immediate; a timelock and a 2-of-3 Safe are planned before real funds.
- **ERC-1155 callbacks:** receiver hooks on the pool, market and desk are re-entry points and must stay behind reentrancy guards.
- **Off-chain trust:** physical custody, grading and price providers are outside the contracts; the design limits but cannot remove that trust.
- **Before mainnet:** independent audit, timelock, bug bounty, monitoring of oracle deviation and pool health, and confirmed crop parameters.

---

7. Status

Deployed on Sepolia on 8 October 2026 with all roles held by the Safe 0xDa15…de12, and live at **agribridge-lilac.vercel.app**. The first lot, 122 kg of rice graded B, was verified through the Safe on the live network. The protocol is a prototype and has not been audited.